Change Point Detection

13st October, 2026

Skoltech

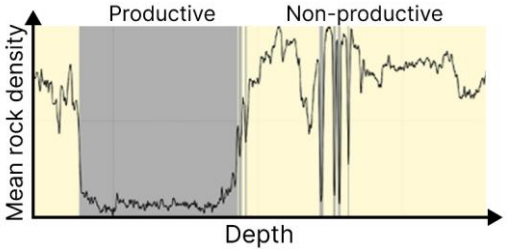
Evgenia Romanenkova,
PhD, Head of Research group in LARSS

What is Change Point Detection?

CPD models detect time steps where the **data-generating distribution changes**, transitioning the process into a new regime.

Geophysics & Drilling[1]

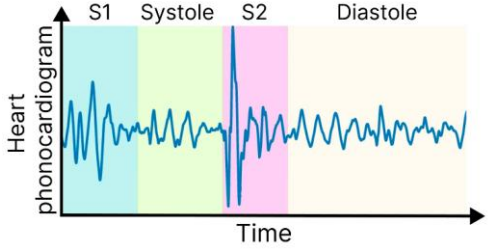
Rock Layer Transition



Change: mean/variance of rock density;
Goal: steer drill bits into productive zones.

Biomedical signals

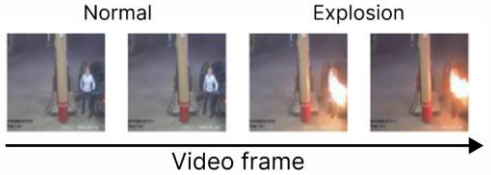
Heart Sound Cycle



Change: frequency, oscillatory dynamics;
Goal: automatic phase segmentation.

Computer Vision

Video Surveillance



Change: visual feature distribution;
Goal: real-time alarm on accidents.

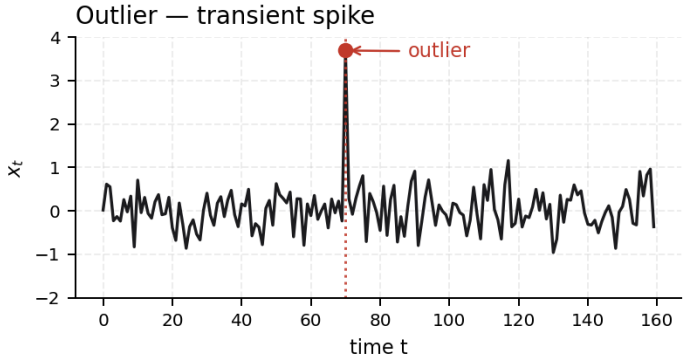
Skoltech

[1] Romanenkova E. et al. “Real-time data-driven detection of the rock-type alteration during a directional drilling”
[2] Chen Y. et al. “End-to-end heart sound segmentation using deep convolutional recurrent network”, Complex & Intelligent Systems, 2021
[3] Romanenkova E. et al. “InDiD: Instant Disorder Detection via a Principled Neural Network”, ACM MM, 2022

Outlier vs Change Point

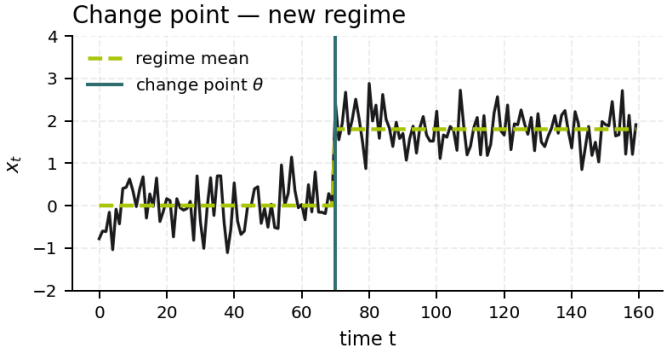
Outlier

- An **isolated**, weird observation x_t;
- **Transient**: does not affect future data generation.
- *Q: Is this specific point unlike?*



Change Point

- A moment when the data-**generating process** changes;
- **Persistent:** Resets the notion of "normal" for all future points
- *Q: Did the underlying mechanism switch regimes?*



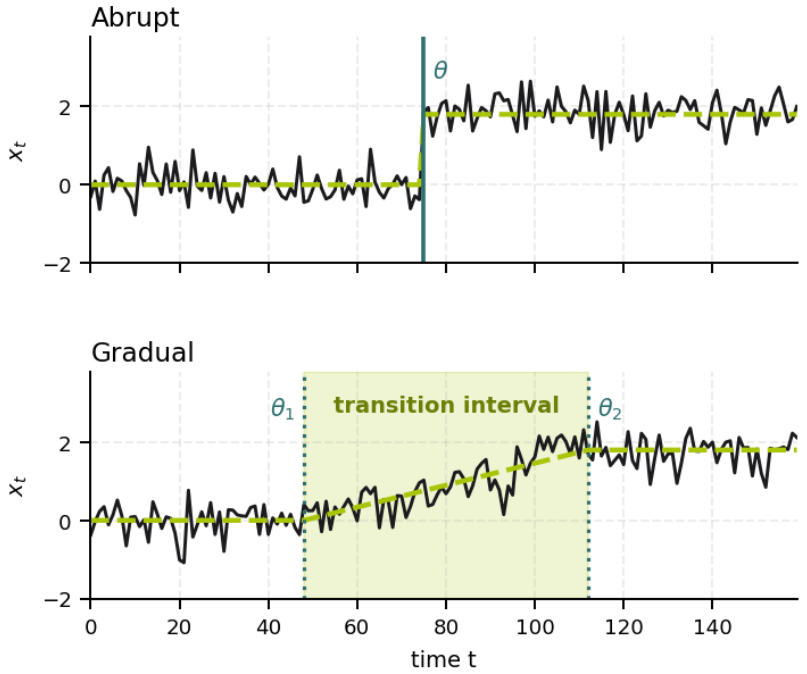
CPD taxonomy

CPD is not one algorithm - it is a family of distinct problems.

1. Nature of change

Abrupt - Instantaneous parameter change;
Good for classical detectors

Gradual - Smooth evolution over an interval



Skoltech

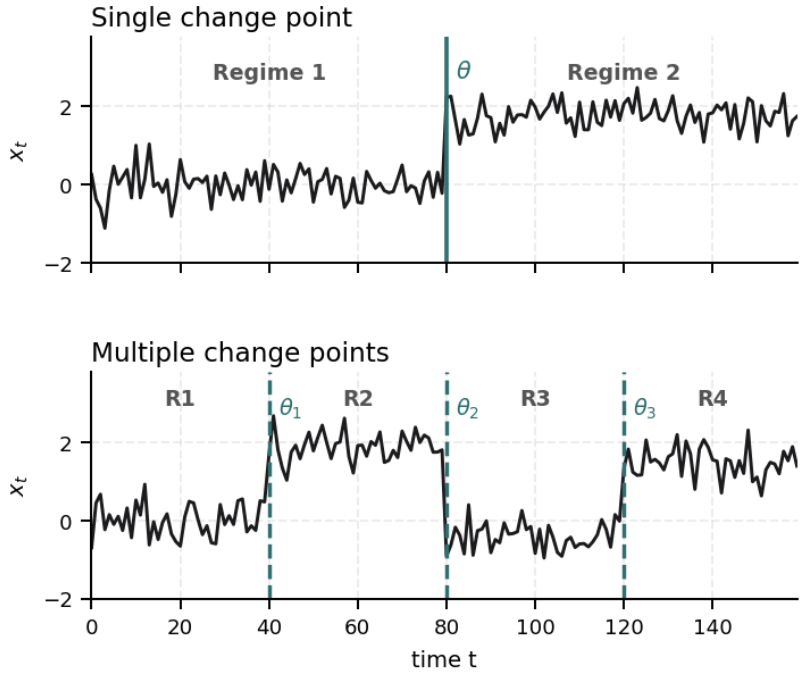
CPD taxonomy

CPD is not one algorithm - it is a family of distinct problems.

2. Number of regimes

Single - Catch the first critical failure;

Multiple - Full time-series segmentation into homoge-nous segments



Skoltech

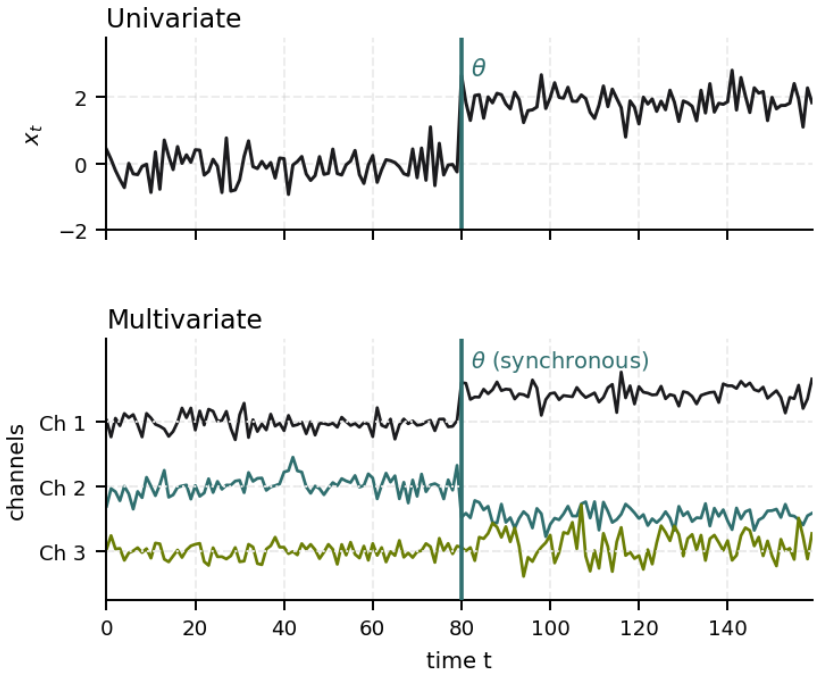
CPD taxonomy

CPD is not one algorithm - it is a family of distinct problems.

3. Dimensionality

Univariate;

Multivariate
Watch out for curse of dimensionality.

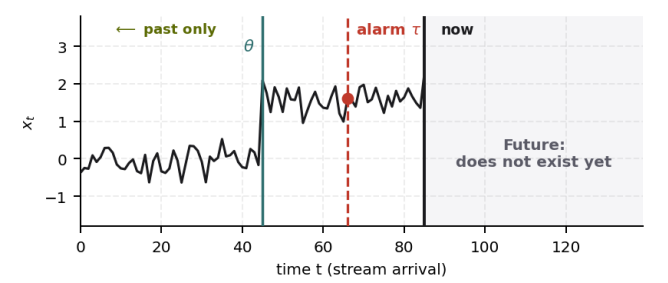


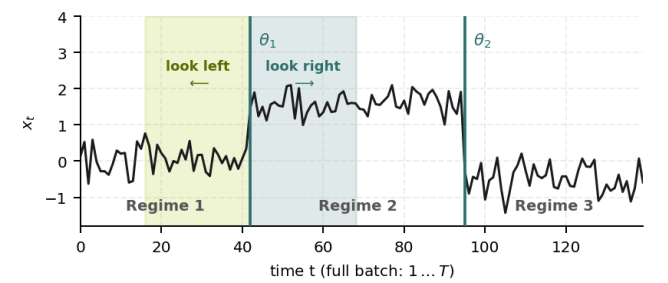
Skoltech

QUIZ TIME!

Offline vs Online

	Online – Quickest detection	Offline – Retrospective
Data	Points arrive as a live stream.	The full sequence is already available.
Causality	Strictly causal. The future does not exist yet.	Non-causal. You can look left and right of any point.
Goal	At every step, make a decision: Sound the alarm or wait?	Reconstruct the complete history of regime changes after the fact.





Our focus for today's lecture

Formalizing the problem

We observe data $x_1, x_2, \ldots$ - univariate or multivariate.
There exist K unknown change points $1 = \theta_0 < \theta_1 < \cdots < \theta_K < \theta_{K+1} = \infty$:

$$x_t \sim f_i \ \ for \ \ t \in [\theta_{i-1}, \theta_i) \ \ and \ \ f_i \neq f_{i+1} \ \ \ \forall i = 1, \ldots, K+1$$

	Online – Quickest detection	**Offline – Retrospective**
Unknown	True next (if multiple) change time θ	Number of change points K (optional) and their location $\theta_1, \ldots, \theta_K$
Output	Alarm stopping time τ	The partition vector $(\tau_1, \ldots, \tau_K)$
Decision Rule	The earliest alarm time: $\inf\{n : Score(x_1 : x_n) \geq h\}$	Minimize total segment cost: $\min_{\tau_1 < \ldots < \tau_K} \sum_{i=1}^{K+1} Cost\,(x_{\tau_{i-1}} : x_{\tau_i})$

Skoltech

Two key metrics for CPD

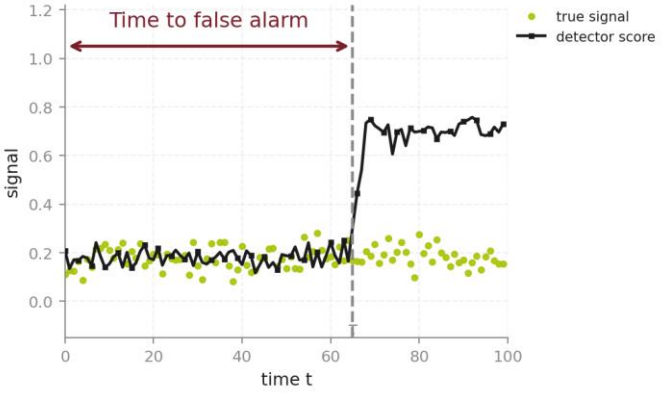
What if the regime **never** changes ($\theta = \infty$)?

- We want the detector to run as long as possible without alarming, i.e.
- Maximize Average Running Length (ARL) ≈ **Time to False Alarms**:

$$ARL_0 = \mathbb{E}_{\theta=\infty}(\tau) \to \infty$$

- Alternatively, we want to minimize the **Probability of False Alarm**:

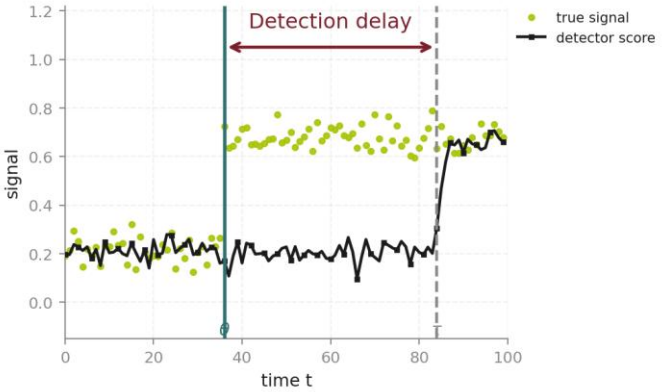
$$PFA = \mathbb{P}(\tau < \theta) \to 0$$



What if the change occurs at time θ?

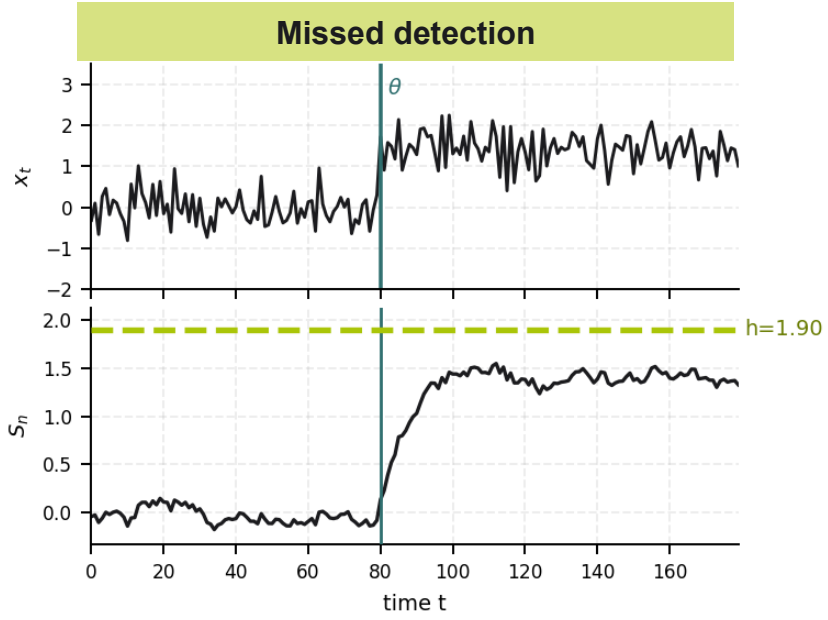
- We want the detector to alarm as fast as it possible, i.e.
- Minimize **Detection Delay**:

$$DD = (\tau - \theta)^+ = \max(0, \tau - \theta)$$



Skoltech

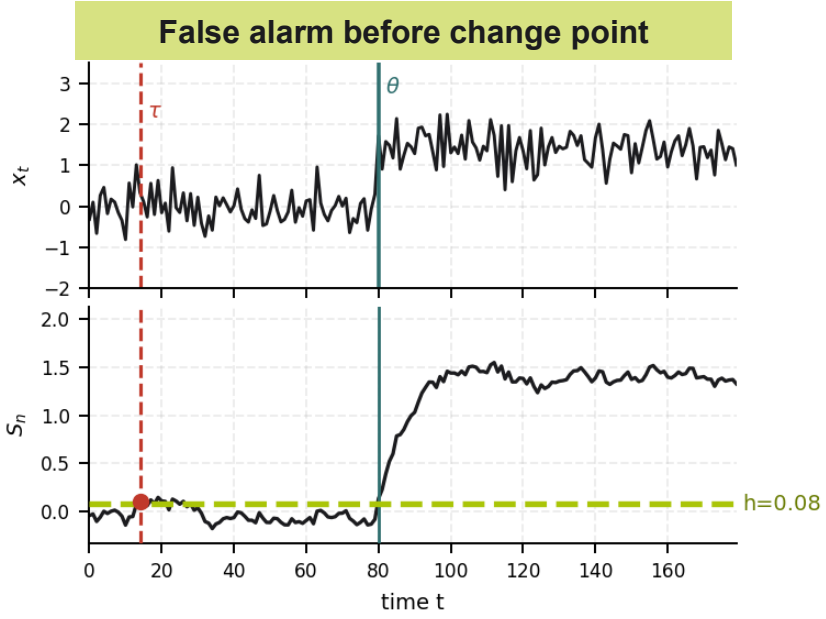
Good detection is always trade-off



Skoltech

Quickest detection = manage the delay <-> false-alarm Pareto frontier.

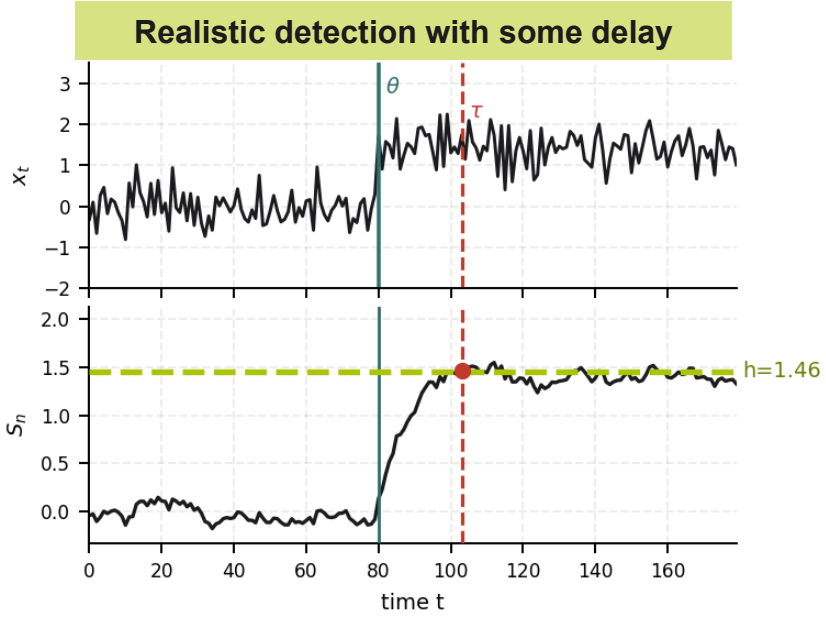
Good detection is always trade-off



Skoltech

Quickest detection = manage the delay <-> false-alarm Pareto frontier.

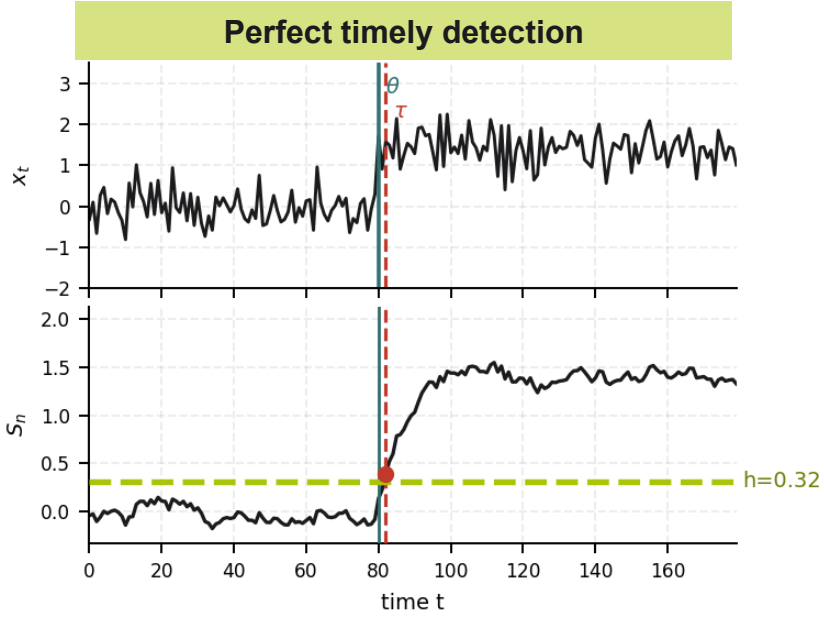
Good detection is always trade-off



Skoltech

Quickest detection = manage the delay <-> false-alarm Pareto frontier.

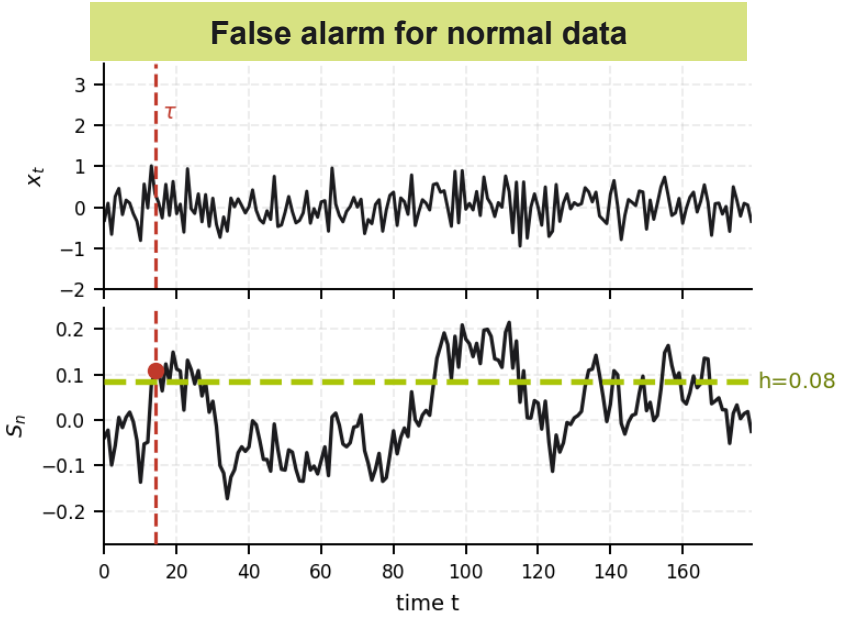
Good detection is always trade-off



Skoltech

Quickest detection = manage the delay <-> false-alarm Pareto frontier.

Good detection is always trade-off



Skoltech

Quickest detection = manage the delay <-> false-alarm Pareto frontier.

Hypothesis testing view for CPD

We observe data $x_1, x_2, \ldots$ - univariate or multivariate.

Two hypothesis:

$$H_0: \forall n \Rightarrow x_{1:n} \sim f_0.$$
$$H_1: \exists k \Rightarrow x_{1:k-1} \sim f_0 \;\&\; x_{k:n} \sim f_1$$

H_1 is a composite hypothesis because the true change point is unknown.

Practical CPD	Statistical Theory analogue
False Alarm	Type I Error
Detection Delay	Type II Error
$\min DD$ $subject\ to\ ARL_0 \geq \alpha$	$\max Power\ (1-\beta)$ $subject\ to\ Type\ I\ Error \leq \alpha$

Classic Online Change Point Detection

Skoltech

The general online skeleton

The modular pipeline behind every sequential change point detector:



The details matter:

1. *How to extract a change signal from an observation?* – raw residuals, statistical likelihood, kernel distances, or neural embeddings.
2. *How to accumulate statistic S_k?* - memoryless, moving average, or cumulative sum with reset.
3. *What is a decision rule? How to set threshold?* – calibrate analytically under $H_0(ARL_0)$ or tune empirically on validation streams.
4. *What should we do after the alarm?* – stop and reset statistic $(S_k \to 0)$, recalibrate predictions.

Shewhart Chart & Hotelling's T^2

Test the current batch, then forget!

Shewhart Chart (R^1) or Hotelling algorithm (R^d):

1. Divide stream into non-overlapping blocks (batches) of size N
2. Compute batch sample mean $\bar{X}_k = \frac{1}{N}\sum_1^N x_{(k-1)N+i}$
3. Standardize against baseline

$$S_k = \frac{\bar{X}_k - \mu_o}{\sigma_0/\sqrt{N}} \text{ or } T_k^2 = N(\bar{X}_k - \mu_0)^T \Sigma_0^{-1} (\bar{X}_k - \mu_0)$$

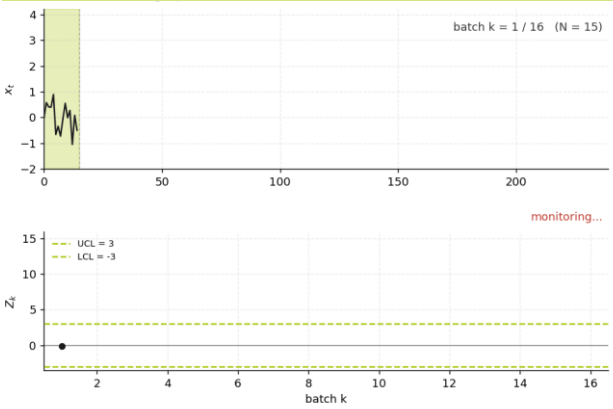
4. Decision Rule

$$\tau = \inf\{k: |S_k| > 3\} \text{ or } \inf\{k: T_k^2 > \chi^2_{d,1-\alpha}\}$$

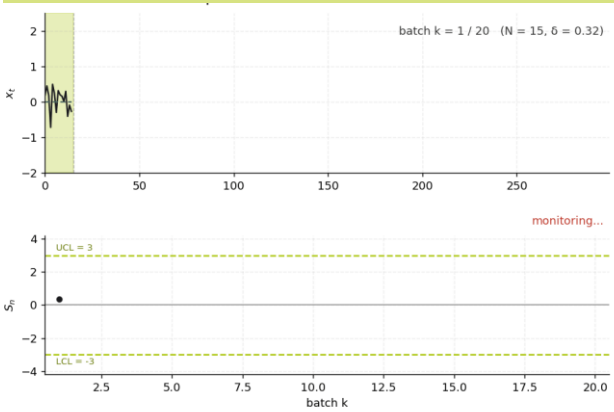
Limitations:

1. No memory between batches;
2. Need estimate of μ_0, σ_0 / Σ_0 for baseline;
3. Completely blind to small, persistent shifts.

Shewhart Chart is good for huge jumps!



But is blind to small shifts



EWMA – geometric memory

Weight the past, trust the present

EWMA algorithm with hyperparameters λ and L:

1. Start from baseline $S_0 = \mu_0 \;\; or \; S_0 = x_1$
2. Update at each point $S_k = (1 - \lambda)\, S_{k-1} + \lambda x_k$
3. Decision Rule $\tau = \inf\{k: |S_k - \mu_0| > L\sigma_s\}$, where $\sigma_s = \sigma_0\sqrt{\lambda/(2 - \lambda)}$.

Role of λ:

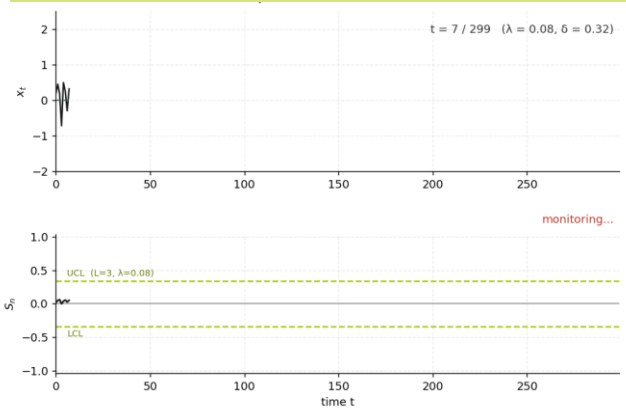
$\lambda \to 1$ – EWMA → Shewhart chart, good for huge jumps;

$\lambda \ll 1$ – long-memory, good for small shifts.

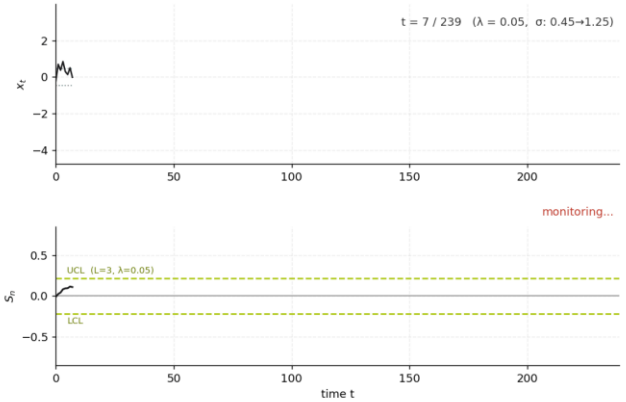
Limitations:

1. Mostly a heuristic smoother;
2. Need estimate of μ_0;
3. Sensitive to hyperparameters.

EWMA can detect even small mean shifts



But not suitable for variance shifts



Log-Likelihood Ratio is a key

Recap hypothesis view of CPD:

$H_0: \forall n \Rightarrow x_{1:n} \sim f_0.$

$H_1: \exists k \Rightarrow x_{1:k-1} \sim f_0 \ \& \ x_{k:n} \sim f_1$

Assume, we know k, f_0, f_1 and all x_i are independent. **How can one choose between f_0 and f_1?**

Neyman-Pearson's Lemma. To maximize detection power at a fixed false alarm rate α, the decision rule must use **likelihood ratio**:

$$\Lambda = \frac{L(Data \mid H_1)}{L(Data \mid H_0)}$$

Actually, k is unknown, so we get:

$$\Lambda_{\mathrm{k}} = \prod_{i=k}^{n} \frac{f_1(x_i)}{f_0(x_i)}$$

To find optimal k: $\max_k \Lambda_k$.

Log-Likelihood Ratio as a drifting Random Walk

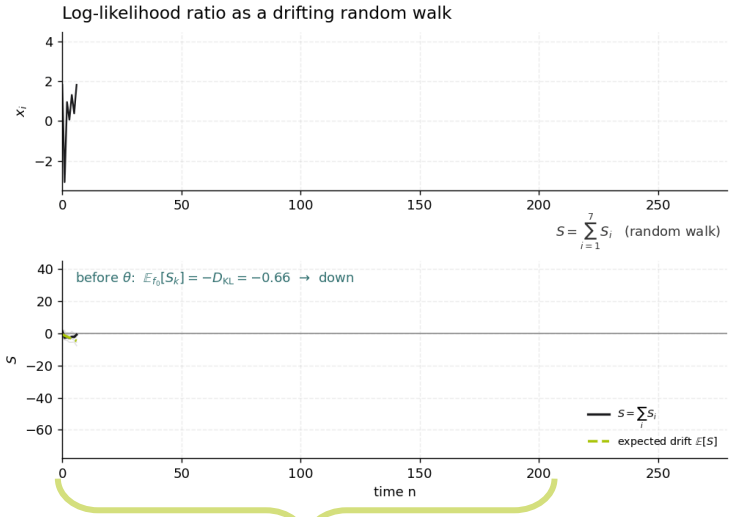
Moving to logarithm, get our statistic:

$$\Lambda_k = \prod_{i=k}^{n} \frac{f_1(x_i)}{f_0(x_i)} \rightarrow S = \log \Lambda_k = \sum_{i=k}^{n} \log \frac{f_1(x_i)}{f_0(x_i)} = \sum_{i=k}^{n} S_i$$

S is a **Random Walk**:

$\mathbb{E}_{f_0} S_i = -D_{KL}(f_0 || f_1) < 0$ – walks down

$\mathbb{E}_{f_1} S_i = D_{KL}(f_1 || f_0) > 0$ – walks up



The cost of debt:
After the true change ($\theta = 120$), the walk wastes ~80 steps just to climb back to 0.

How to eliminate accumulated past debt?

Skoltech

Page's CUSUM

The gold standard of online CPD since 1954[1]!

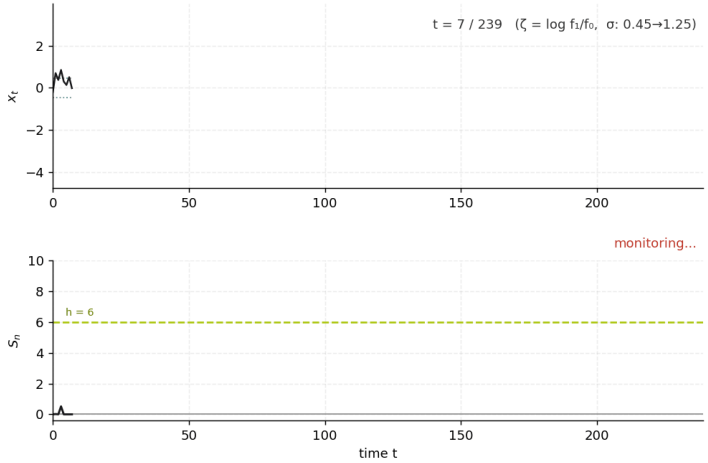
CUSUM algorithm with hyperparameter h:

1. Initialize $S_0 = 0$
2. Update at each point $\mathrm{S_k} = \max(0,\ S_{k-1} + \log\frac{f_1(x_k)}{f_0(x_k)})$
3. Decision Rule $\tau = \inf\{k: S_k \geq h\}$

In practice, we run two parallel charts $\mathrm{S_k^+}$ and $\mathrm{S_k^-}$, then:

$$\tau = \inf\{k: \max(\mathrm{S_k^+}, \mathrm{S_k^-}) \geq h\}$$

CUSUM detects diverse changes



CUSUM optimizes worst-case detection delay subject to $ARL_0 \geq \gamma$

Key limitation:
Requires exact knowledge of f_0, f_1, but GLR-CUSUM[2] relaxes this by estimating parameters via MLE ($O(n)$ per step).

Skoltech

[1] Page E., "Continuous inspection schemes", Biometrika, 1954
[2] Lorden G., "Procedures for reacting to a change in distribution", The Annals of Mathematical Statistics, 1971

CUSUM vs Shiryaev-Roberts

CUSUM is **pessimistic**, tracking the single most likely change point to minimize the worst-case delay.

Instead, Shiryaev and Roberts proposed integrating evidence across all past time steps to minimize average delay.

	CUSUM	Shiryaev-Roberts
Aggregation	the MAXIMUM over all k: $$S_n = \log \max_{1 \le k \le n} \prod_{i=k}^{n} \frac{f_1(x_i)}{f_0(x_i)}$$	the SUM over all k: $$S_n = \sum_{k=1}^{n} \prod_{i=k}^{n} \frac{f_1(x_i)}{f_0(x_i)}$$
Recursion	$$S_n = \max(0, S_{n-1} + \log \frac{f_1(x_n)}{f_0(x_n)})$$	$$S_n = (1 + S_{n-1}) \cdot \frac{f_1(x_n)}{f_0(x_n)}$$
Optimality	Worst-case delay (Lorden's minimax criterion)	Average delay (Pollak's criterion)
Nature	Frequentist	Bayesian

Skoltech

Shiryaev A., "Stochastic Change-Point Detection Problems", MCME, 2017

Breaking the parametric barrier

Let us again recall hypothesis view of CPD:

$H_0: \forall n \Rightarrow x_{1:n} \sim P.$

$H_1: \exists k \Rightarrow x_{1:k-1} \sim P \;\&\; x_{k:n} \sim Q$

Assume for now, we **DON'T** know P, Q and cannot compute the log-likelihood ratio.

How can one choose between them?

Kernel Two-Sample Test. Map data into an RKHS $\mathcal{H}$ with $\phi(x) = k(x, \cdot)$. Represent distributions as mean embeddings $\mu_P = \mathbb{E}_{x \sim P}[\phi(x)]$ and $\mu_Q = \mathbb{E}_{x \sim Q}[\phi(x)]$. Then:

$$\mathbf{MMD^2}(\boldsymbol{P}, \boldsymbol{Q}) = \left\| \boldsymbol{\mu_P} - \boldsymbol{\mu_Q} \right\|^2_{\boldsymbol{\mathcal{H}}}.$$

$\mathrm{MMD}(P, Q)$ can be approximated with simple sum of kernel values over pairs of real points:

$$\widehat{\mathrm{MMD}}^2(P, Q) = \frac{1}{n^2} \underbrace{\sum_{i,j} k(x_i, x_j)}_{\text{Similarity in } P} - \frac{2}{n^2} \underbrace{\sum_{i,j} k(x_i, y_j)}_{\text{Cross-similarity}} + \frac{1}{n^2} \underbrace{\sum_{i,j} k(y_i, y_j)}_{\text{Similarity in } Q},$$

where $x \sim P, y \sim Q$.

Gretton A. et al. "A kernel two-sample test", JMLR, 2012

Kernel CPD

The original Two-Sample Test compares two static distributions P and Q.
In Online CPD, we compute the discrepancy dynamically across two adjacent **sliding windows**:
$W_{ref} = \{x_{t-2w+1}, \dots, x_{t-w}\}$ and $W_{curr} = \{x_{t-w+1}, \dots, x_t\}$.

Kernel CPD[1] algorithm with hyperparameter w:

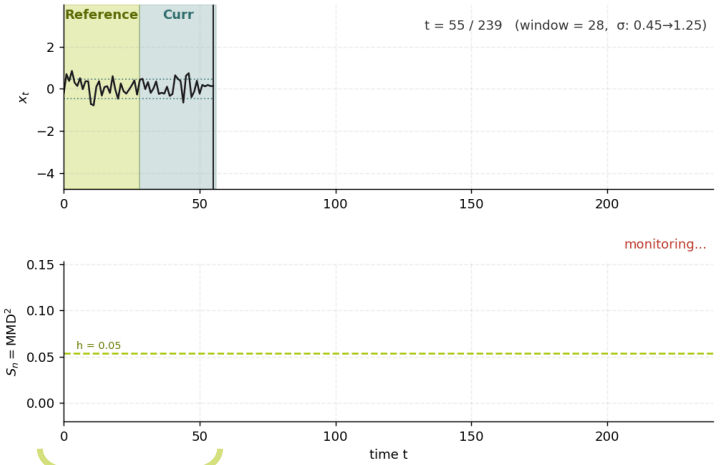
1. Collect initial buffer of $2w$ observations;
2. Update $S_k = \widehat{\mathrm{MMD}}^2(W_{ref}, W_{curr})$
3. Decision Rule $\tau = \inf\{k : S_k > h\}$

As $w \to \infty$, $Power_{KernelCPD} \to 1$.

Limitations:

1. Cannot detect changes faster than w;
2. Sensitive to kernel choice;
3. Computing pairwise kernel matrix costs $O(w^2)$ per step – reduced to $O(\log n)$ in 2025[2].

Non-parametric Kernel CPD can also detect diverse changes



All window-based methods have natural detection delay of window-size.

[1] Li S. et al. “M-Statistic for Kernel Change-Point Detection”, NeurIPS, 2015
[2] Kalinke F. et al. “Optimal Online Change Detection via Random Fourier Features”, NeurIPS, 2025

Modern Change Point Detection

Skoltech

The dilemma of high-dimensionality: Why naive scaling fails?

Suppose we monitor a system with $d = 1000$ sensors. There are two naive approaches:

Variant 1. Feed $x_t \in \mathbb{R}^d$ as a single vector.

- Covariance matrix bottleneck:
  Estimating Σ^{-1} requires $O(d^2)$ params and fails if $N \ll d$ due to singularity.
- Sparse shifts:
  If 2 break, 998 noisy sensors drown the signal → $O(\sqrt{d})$ detection delay.
- Distance concentration:
  $\|x_i - x_j\| \approx const$ → RBF kernel flat → MMD loses power.

Variant 2. Run d parallel detectors.

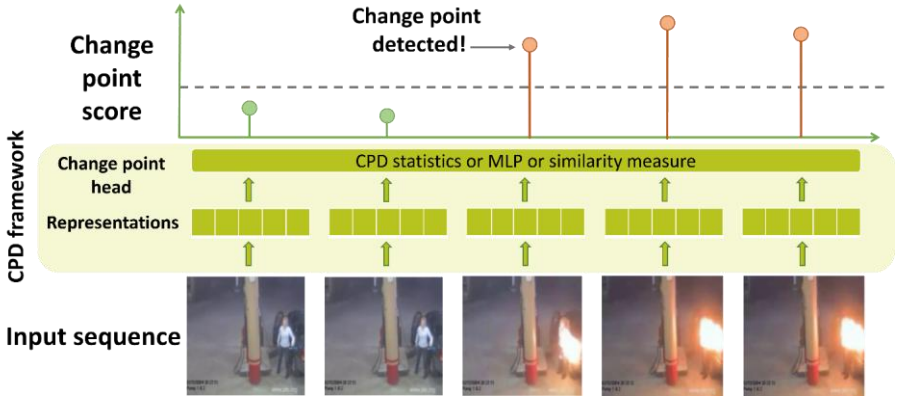
- Multiple testing trap:
  Even if each sensor has $\alpha = 1\%$ → $\mathbb{P}(Alarm) = 1 - (1 - \alpha)^d \approx 99.99\%$.
- Blind to correlation shifts:
  Cannot detect breaks in covariance when individual sensors remain normal.
- Heterogeneous data: different dimensions can have different types (continuous, discrete).

Raw features fail. We must use **representation learning** to project $x_t \in \mathbb{R}^d$ into a compact latent manifold $z_t \in \mathbb{R}^m$, where $m \ll d$ and simple CPD methods can effectively work.

InDiD = Classic CPD criteria + Deep learning

Let's formulate CPD as a binary classification problem:
"Is time step t a change point or not?" and train a deep end-to-end model.

❌ **The Binary Cross-Entropy (BCE) challenge**:
BCE treats time steps as **independent** points, completely **ignoring** true **CPD objectives**: Detection Delay & False Alarm Rate!



✅ Let's combine these objectives into a loss function for the neural network:

$$\mathcal{L}_{InDiD} = \mathcal{L}_{Delay} - c\mathcal{L}_{FA}$$

Through backpropagation, gradient descent adapts the representations:

- $\mathcal{L}_{Delay}$ - forces the model to react faster to true changes;
- $\mathcal{L}_{FA}$ - rewards the model for staying quiet during normal operation.

Skoltech

Romanenkova E. et al. "InDiD: Instant Disorder Detection via a Principled Neural Network", ACM MM, 2022

InDiD in action: Real-world video surveillance

Results for extremely high-dimensional CPD in video surveillance:

- Explosions – abrupt change as the flash changes the intensity of the frame;
- Road accidents – more gradual change since the content of the frame changes.

Dataset	Method	Mean Time to FA ↑	Mean DD ↓	F1 ↑
Explosion (Abrupt)	KL-CPD	15.64 ± 0.18	0.25 ± 0.04	0.279 ± 0.038
	TSCP	15.07 ± 0.13	0.15 ± 0.01	0.348 ± 0.025
	BCE	14.74 ± 0.47	**0.13 ± 0.03**	0.309 ± 0.088
	InDiD	**15.73 ± 0.11**	0.20 ± 0.04	**0.530 ± 0.048**
Road accidents (Gradual)	KL-CPD	15.35 ± 0.24	0.91 ± 0.08	0.106 ± 0.046
	TSCP	**15.26 ± 0.53**	0.67 ± 0.05	0.235 ± 0.029
	BCE	14.43 ± 0.17	**0.53 ± 0.10**	0.176 ± 0.065
	InDiD	13.83 ± 0.54	0.66 ± 0.08	**0.287 ± 0.028**

InDiD almost **doubles the F1 score** on explosions (**0.31**→ **0.53**) and road accidents (0.18 → 0.29) compared to standard BCE!

Skoltech

Romanenkova E. et al. “InDiD: Instant Disorder Detection via a Principled Neural Network”, ACM MM, 2022

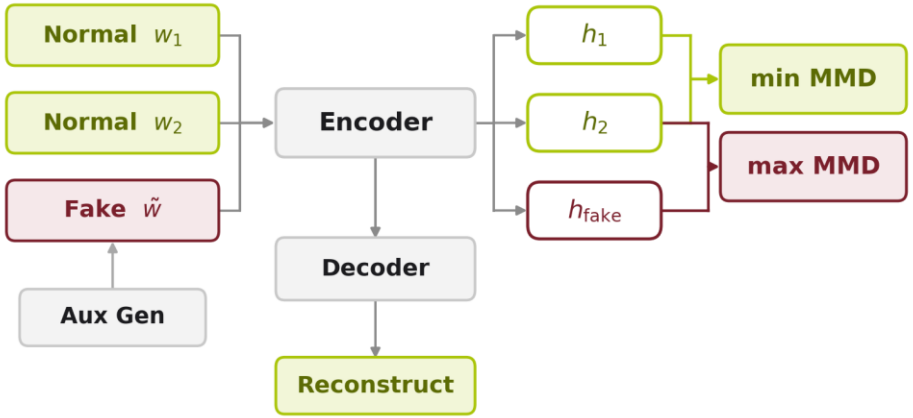
KL-CPD = Kernel CPD + Deep Learning

KL-CPD **parametrizes kernel** through RNN $g_\phi(\cdot)$:

$$k_\theta(x, y) = \exp\left(-\frac{\left\|g_\phi(x) - g_\phi(y)\right\|^2}{2\sigma^2}\right)$$

Three training tasks:

- Reconstruction loss: RNN Autoencoder learns to preserve underlying dynamics;
- False alarm control: $\min MMD\ (h_1, h_2)$ keeps small distance during normal operation;
- Test power control: $\max MMD\ (h_2, h_{fake})$ pushes normal data apart from synthetic anomalies.



Why Auxiliary Generator?
Change points are rare: the generator creates synthetic samples so the kernel can learn separating regimes.

Online detector: $\tau = \inf\{t: \widehat{MMD}^2(h_{ref}, h_{curr}) > thr\}$

Chang W. et al. “Kernel Change-point Detection with Auxiliary Deep Generative Models”, ICLR, 2019

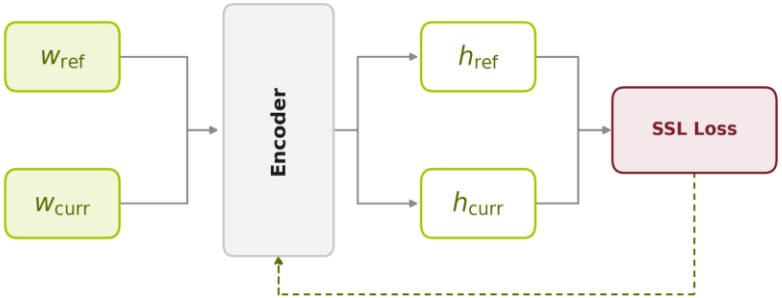
Self-Supervised Learning meets CPD

Forget about complex CPD principles. All we need is to **learn a good encoder** $g_\phi(\cdot)$ that makes **normal data look similar, and makes regime changes look different**.

Different Self-Supervised Learning models help:

- CPC[1] predicts future latent vectors from past context via InfoNCE;
- TS-BYOL[2] - dual RNNs (online & momentum target) trained on augmentations;
- TS2VEC[2] contrasts representations through hierarchical levels.

Methods don't require any labeled data!



All detection logic is **embedded into the encoder**, so we can use simple cosine distance (or Euclidean) as the decision rule:

$$\tau = \inf\{t: (1 - cos(h_{ref}, h_{curr})) > thr\}$$

Skoltech

[1] Deldari S. et al. "Time series change point detection with self-supervised contrastive predictive coding", WWW, 2021
[2] Bazarova A. et al. "Normalizing Self-Supervised Learning for Provably Reliable Change Point Detection", ICDM, 2024

Controlling SSL latent geometry for CPD

Unconstrained SSL encoder can lead to two issues:

- Distance collapse ($L_1 \to 0$): different regimes are squashed into the same point $\Longrightarrow$ **missed changes**;
- Distance explosion ($L_2 \to \infty$): harmless sensor noise is amplified $\Longrightarrow$ **false alarms**.

The solution - add Bi-Lipschitz mapping $\mathcal{X} \to \mathcal{H}$ via **Spectral Normalization** (SN):

$$L_1\|x - x'\|_{\mathcal{X}} \leq \|h(x) - h(x')\|_{\mathcal{H}} \leq L_2\|x - x'\|_{\mathcal{X}}$$

This approach **provably preserves statistical test power** and Type-II error convergence rate for different CPD heads.

Adding SN consistently improves detection across all base SSL models.

Better representations yields better detector: SN-TS2Vec sets state-of-the-art results across all datasets.

Method	Yahoo, F1	USC-HAD, F1	HASC, F1
KL-CPD	0.544	0.743	0.544
TS-CP2	0.843	0.824	0.843
TS-BYOL	0.768	0.500	0.768
TS2Vec	0.816	0.873	0.816
SN-TS-BYOL	0.789	0.500	0.789
SN-TS2Vec	**0.852**	**0.909**	**0.852**

Bazarova A. et al. “Normalizing Self-Supervised Learning for Provably Reliable Change Point Detection”, ICDM, 2024

Change Point Detection nowadays

Classical CPD remains irreplaceable for real-time systems

Classical CPD

- Exact false-alarm guarantees;
- $O(1)$ streaming latency per observation.

Deep CPD

- Black-box failure modes;
- Heavy inference latency & GPU overhead.

1. **Real-time A/B testing and release guardrails[1]:**
   The problem: Product managers peek at dashboards 20 times a day, inflating the false alarm rate up to 30%;
   The solution: Sequential probability ratio testing enables anytime monitoring without sample-size penalties, stopping broken releases within hours while strictly controlling Type-I error ($\alpha \leq 5\%$).

2. **High-throughput cloud telemetry[2]:**
   The problem: In high-volume cloud infrastructure, neural models with millisecond latency create severe queue backpressure and memory crashes.
   The solution: Functional Pruning CUSUM processes streaming metrics in guaranteed $O(1)$ or $O(\log n)$ time on a single CPU thread.

Skoltech

[1] Kurennoy A. et al. “YEAST: Yet Another Sequential Test”, NeurIPS, 2025
[2] Ward K. et al. “A constant-per-iteration likelihood ratio test for online changepoint detection for exponential family models”, Statistics and Computing, 2024

The new frontiers of CPDs: LLMs and Agents

Today, the most exciting frontier is the intersection of sequential detection and Generative AI.

Monitoring LLM APIs[1]
Compare first-token log-probabilities on a fixed short prompt with a permutation test. The method detects minute model shifts at lower cost.

Segmenting watermarked LLM text[2]
Compute watermark p-values over rolling windows, the sequence of p-values is a target stream for CPD. Then run nonparametric CPD on that sequence.

In-context CPD inside transformers[3]
Transformers can adapt in-context by detecting a change in the prompt. PE can encode change point side information.

CPD agents[4]
Combine an ensemble of classic CPD methods into self-evolving LLM agent for autonomous financial time-series CPD.

[1] Li X. et al. “Segmenting watermarked texts from language models”, NeurIPS, 2024
[2] Chauvin T. et al. “Log probability tracking of LLM APIs”, ICLR, 2026
[3] Dudley C. et al. “In-Context Learning Under Regime Change”, ICML Workshop FMSD, 2026
[4] Jiang L. et al. “EvoTS-Agent: A Self-Evolving LLM Agent for Financial Time Series Change Point Detection”, arXiv:2608.17933, 2026

Conclusions

1. CPD is a family of problems, not one algorithm. Before choosing a method, fix the setting: type of shifts, abrupt vs gradual, single or multiple, univariate vs multivariate;
2. Online CPD is a delay–false-alarm trade-off. Design for the Pareto frontier you can afford.
3. There are two useful views on CPD problem: criteria-oriented and hypothesis-testing – both are valid and complementary.
4. Most CPD methods share one pipeline: score → accumulate → threshold.
5. Classical sequential tests remain the fundament of CPD and are still strong in many real-time systems.
6. Deep CPD methods extends the classic CPD idea and pipeline to complex high-dimensional data.
7. A new frontier is CPD + generative AI.

Appendix

Summary of considered methods

	Evidence measure	Memory accumulation	Decision rule
Shewhart chart	Batch mean deviation	None	$S_n \geq 3\sigma/\sqrt{N}$
EWMA	Raw value x_n	Geometric moving average	$S_n \geq L\sigma_0\sqrt{\lambda/(2-\lambda)}$
CUSUM	Log-likelihood ratio, $\log\frac{f_1}{f_0}$	Cumulative sum with zero-floor	$S_n \geq h$
Shiryaev-Roberts	Log-likelihood ratio, $\log\frac{f_1}{f_0}$	Mixture over all past change times	$S_n \geq h$
Kernel CPD	Maximum Mean Discrepancy	Sliding window buffer	$MMD^2 \geq h$
InDiD	Deep feature representation	Recurrent neural state	$p_n \geq h$
KL-CPD	MMD in deep latent space	Sliding window buffer	$MMD^2 \geq h$
SSL-CPD	Cosine distance (or any) between SSL embeddings	Sliding window buffer	$cos(e_{t-1}e_t), \geq h$

Skoltech

Summary of the current CPD area stage

Data dim.	<10	10-100	>100
Method	Traditional CPD methods	Traditional or ML/DL CPD methods	
Current research gaps	Strict assumptions: • fixed prior data distribution; • pre-define kernel shape or similarity measure.	• Have a moderate quality on data with higher dimensionality; • The detection delay is of the order of $\sqrt{d}$, where d – data dimensionality; • Noise features can jam the change-signal.	Classic-alike approaches work in strict assumptions: • concrete data models; • sparsity assumption. There is a lack of representation-based methods for CPD. Current approaches still mostly focus on low-dimensional time series.
	Need adaptation for real-world time series		